

```

1 import pandas as pd # This is used to working with data.
2 import numpy as np # This is used for working with numbers and arrays.
3 import matplotlib.pyplot as plt # This line of code is used for visualisation or drawing graphs.

```

```

1 from sklearn.model_selection import train_test_split # These two lines of code is for the prediction at the end.
2 from sklearn.linear_model import LinearRegression

```

```

1 df = pd.read_csv("listings.csv.gz") # This code is used to read the file uploaded.
2 print("First five rows:") # checking to see if the file is readable and gives output.
3 print(df.head())

```

First five rows:

	id	listing_url	scrape_id	last_scraped
0	44077	https://www.airbnb.com/rooms/44077	20260621054341	2026-06-21
1	69785	https://www.airbnb.com/rooms/69785	20260621054341	2026-06-21
2	85156	https://www.airbnb.com/rooms/85156	20260621054341	2026-06-21
3	159889	https://www.airbnb.com/rooms/159889	20260621054341	2026-06-21
4	162809	https://www.airbnb.com/rooms/162809	20260621054341	2026-06-21

	source	name
0	city scrape	cosy comfortable Irish cottage twin
1	city scrape	Number Nine : Ballsbridge
2	city scrape	Cosy Comfortable Irish Cottage 1 Double Bed
3	city scrape	Friendly Single Room
4	city scrape	5.5 miles south of Dublin City :^)

	description	neighborhood_overview
0	Our house is a cosy comfortable cottage with ...	NaN
1	Cosy and comfortable, the cottage is a surpris...	NaN
2	Our Cottage is a charming light filled cottage...	NaN
3	Washing can be done at a cost of €5 per load...	NaN
4	A nice place to relax after the bustle of the ...	NaN

	picture_url	host_id	...
0	https://a0.muscache.com/pictures/525706/050a3a...	193005	...
1	https://a0.muscache.com/pictures/446521/4026d7...	205636	...
2	https://a0.muscache.com/pictures/1749253/9ed2a...	193005	...
3	https://a0.muscache.com/pictures/3031697/a8259...	766611	...
4	https://a0.muscache.com/pictures/86694529/c07b...	777681	...

	review_scores_communication	review_scores_location	review_scores_value
0	4.94	4.68	4.83
1	4.76	4.78	4.72
2	4.89	4.64	4.77
3	4.87	4.66	4.71
4	4.97	4.79	4.86

	license	instant_bookable	calculated_host_listings_count
0	NaN	NaN	2
1	NaN	NaN	1
2	NaN	NaN	2

```

3      NaN      NaN      3
4      NaN      NaN      2

calculated_host_listings_count_entire_homes \
0      0
1      1
2      0
3      0
4      0

calculated_host_listings_count_private_rooms \
0      2
1      0
2      2
3      3
4      2

calculated host listings count shared rooms reviews per month

```

```

1 print("\nShape (rows, columns):") # These code is used for understanding the data.
2 print(df.shape)                  # code gives the output of all the rows and columns and gives information about how many rows and columns are there.
3 print("\nColumn information:")
4 print(df.info())
5
6

```

```

Shape (rows, columns):
(6966, 90)

```

```

Column information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6966 entries, 0 to 6965
Data columns (total 90 columns):

```

#	Column	Non-Null Count	Dtype
0	id	6966 non-null	int64
1	listing_url	6966 non-null	object
2	scrape_id	6966 non-null	int64
3	last_scraped	6966 non-null	object
4	source	6966 non-null	object
5	name	6966 non-null	object
6	description	6871 non-null	object
7	neighborhood_overview	0 non-null	float64
8	picture_url	6966 non-null	object
9	host_id	6966 non-null	int64
10	host_url	6966 non-null	object
11	host_profile_id	6966 non-null	int64
12	host_profile_url	6965 non-null	object
13	host_name	6966 non-null	object
14	host_since	0 non-null	float64
15	hosts_time_as_user_years	6966 non-null	int64
16	hosts_time_as_user_months	6966 non-null	int64
17	hosts_time_as_host_years	6966 non-null	int64
18	hosts_time_as_host_months	6966 non-null	int64

```

19 host_location          5027 non-null    object
20 host_about             2892 non-null    object
21 host_response_time     0 non-null      float64
22 host_response_rate     0 non-null      float64
23 host_acceptance_rate   0 non-null      float64
24 host_is_superhost      6966 non-null    object
25 host_thumbnail_url     0 non-null      float64
26 host_picture_url       6966 non-null    object
27 host_neighbourhood     0 non-null      float64
28 host_listings_count    6966 non-null    int64
29 host_total_listings_count 0 non-null      float64
30 host_verifications     0 non-null      float64
31 host_has_profile_pic   6966 non-null    object
32 host_identity_verified 6966 non-null    object
33 neighbourhood          0 non-null      float64
34 neighbourhood_cleansed 6966 non-null    object
35 neighbourhood_group_cleansed 0 non-null      float64
36 latitude               6966 non-null    float64
37 longitude              6966 non-null    float64
38 property_type          6966 non-null    object
39 room_type              6966 non-null    object
40 accommodates           6966 non-null    int64
41 bathrooms              4954 non-null    float64
42 bathrooms_text         6946 non-null    object
43 bedrooms               5062 non-null    float64
44 beds                  5243 non-null    float64
45 amenities              6966 non-null    object
46 price                  5578 non-null    object
47 price_quote_checkin_date 5602 non-null    object

```

```

1 columns = ["price", "room_type", "neighbourhood_cleansed", "accommodates", "bedrooms", "number_of_reviews", "review_scores_rating", "minimum_nights", "availability_3
2 df = df[columns] # These code is used to sort and get the columns that we need for the analysis.
3 print("\nData after keeping only the columns we want:")
4 print(df.head())
5

```

Data after keeping only the columns we want:

	price	room_type	neighbourhood_cleansed	accommodates	bedrooms	\
0	\$93.33	Private room	Dn Laoghaire-Rathdown	2	NaN	
1	\$255.00	Entire home/apt	Dublin City	4	2.0	
2	\$95.33	Private room	Dn Laoghaire-Rathdown	2	NaN	
3	\$67.50	Private room	Dublin City	1	NaN	
4	\$101.00	Private room	Dn Laoghaire-Rathdown	6	2.0	

	number_of_reviews	review_scores_rating	minimum_nights	availability_365
0	404	4.82	1.0	45
1	181	4.79	2.0	24
2	365	4.81	1.0	62
3	493	4.75	1.0	314
4	427	4.86	1.0	12

```

1 df["price"] = df["price"].str.replace("$", "", regex=False) # These lines of codes is used to clean the price column.
2 df["price"] = df["price"].str.replace(",", "", regex=False) # The "regex" function tells the pandas to treat $ as a plain character so we can easily remove it.

```

```

3 df["price"] = pd.to_numeric(df["price"])
4
5 print("\nPrice column after cleaning:")
6 print(df["price"].head())
7
8

```

Price column after cleaning:

```

0    93.33
1   255.00
2    95.33
3    67.50
4   101.00

```

Name: price, dtype: float64

```

1 print("\nMissing values in each column:") # These two lines of code is to print the missing values in each coloumn.
2 print(df.isnull().sum())
3
4 print("\nNumber of duplicate rows:") # These two lines is used to find the number of duplicate rows in the dataset.
5 print(df.duplicated().sum())
6
7 df = df.drop_duplicates() # Code used to remove the duplicate rows.
8 df = df.dropna() # This code is used to remove any rows which is in the column that is needed.
9 print("\nShape after removing duplicates and missing values:") # Printing the values after removing the unwanted values and duplicates.
10 print(df.shape)
11

```

Missing values in each column:

```

price          1388
room_type       0
neighbourhood_cleansed  0
accommodates    0
bedrooms       1904
number_of_reviews  0
review_scores_rating  1097
minimum_nights   1
availability_365  0
dtype: int64

```

Number of duplicate rows:

142

Shape after removing duplicates and missing values:

(3402, 9)

```

1 # These lines of codes is used to remove any strange prices as some listing have prices of zero or very high price therefore we are keeping the prices which make se
2
3 df = df[(df["price"] > 0) & (df["price"] < 1000)]
4 print("\nBasic statistics of the price:")

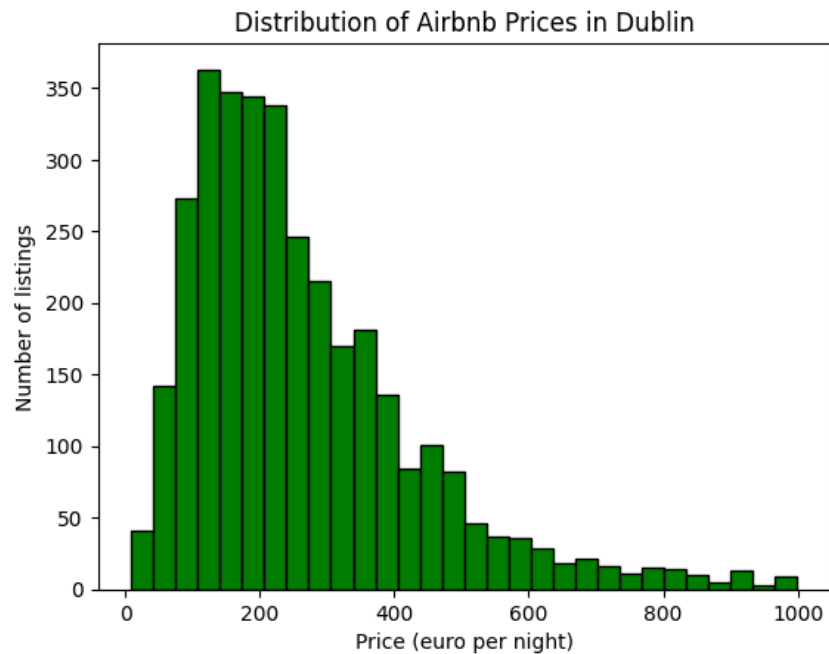
```

```
5 print(df["price"].describe())
6
```

Basic statistics of the price:

```
count    3346.000000
mean      263.561512
std       168.367674
min        8.660000
25%       143.000000
50%       223.000000
75%       342.500000
max       999.000000
Name: price, dtype: float64
```

```
1 # This codes is for visualisation, used a bar graph to represent "Distribution of Airbnb prices in Dublin"
2
3 plt.hist(df["price"], bins=30, color="green", edgecolor="black")
4 plt.title("Distribution of Airbnb Prices in Dublin")
5 plt.xlabel("Price (euro per night)")
6 plt.ylabel("Number of listings")
7 plt.show()
8
```

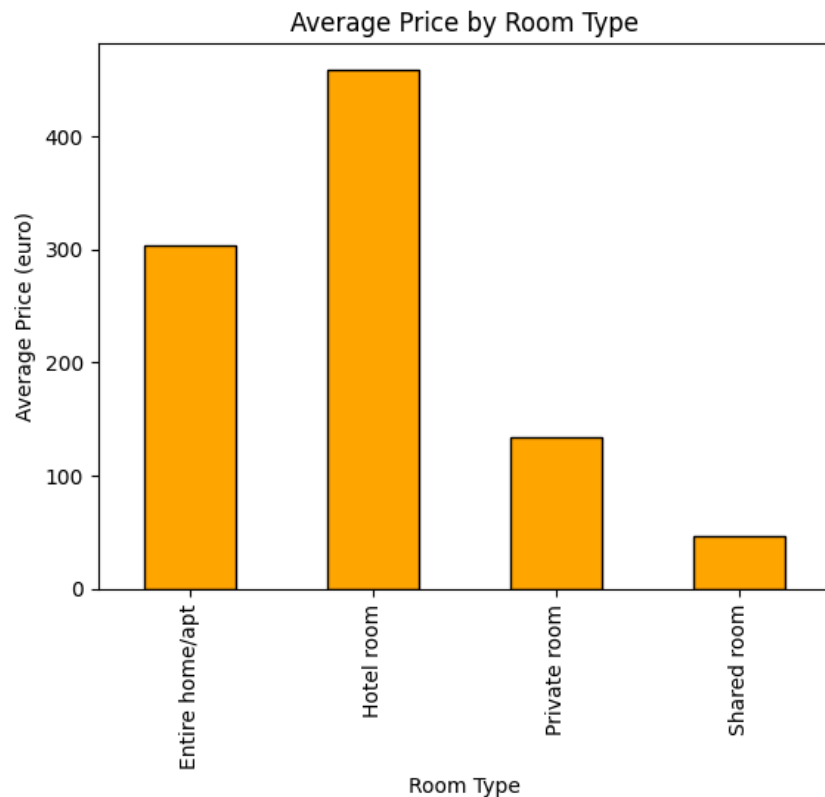


```
1 # This code is used for to find the average price per each room
2
```

```
3 avg_price_room = df.groupby("room_type")["price"].mean()
4 print("\nAverage price by room type:")
5 print(avg_price_room)
6
7 # This code is used for visualisation of average price by room using a bar graph
8
9 avg_price_room.plot(kind="bar", color="orange", edgecolor="black")
10 plt.title("Average Price by Room Type")
11 plt.xlabel("Room Type")
12 plt.ylabel("Average Price (euro)")
13 plt.show()
14
```

Average price by room type:

```
room_type
Entire home/apt    303.124320
Hotel room         459.000000
Private room       133.209552
Shared room        46.095000
Name: price, dtype: float64
```



```
1 # These codes is used to group the average price of top 10 areas.
2
3 avg_price_area = df.groupby("neighbourhood_cleansed")["price"].mean()
4 avg_price_area = avg_price_area.sort_values(ascending=False).head(10)
5 print("\nTop 10 areas by average price:")
6 print(avg_price_area)
7
8 # This code is used for visualisation of top ten areas by average price.
9
10 avg_price_area.plot(kind="bar", color="green", edgecolor="black")
11 plt.title("Top 10 Areas by Average Price")
12 plt.xlabel("Area")
13 plt.ylabel("Average Price (euro)")
14 plt.show()
15
16
```

Top 10 areas by average price:

neighbourhood_cleaned

Dn Laoghaire-Rathdown 277.755737

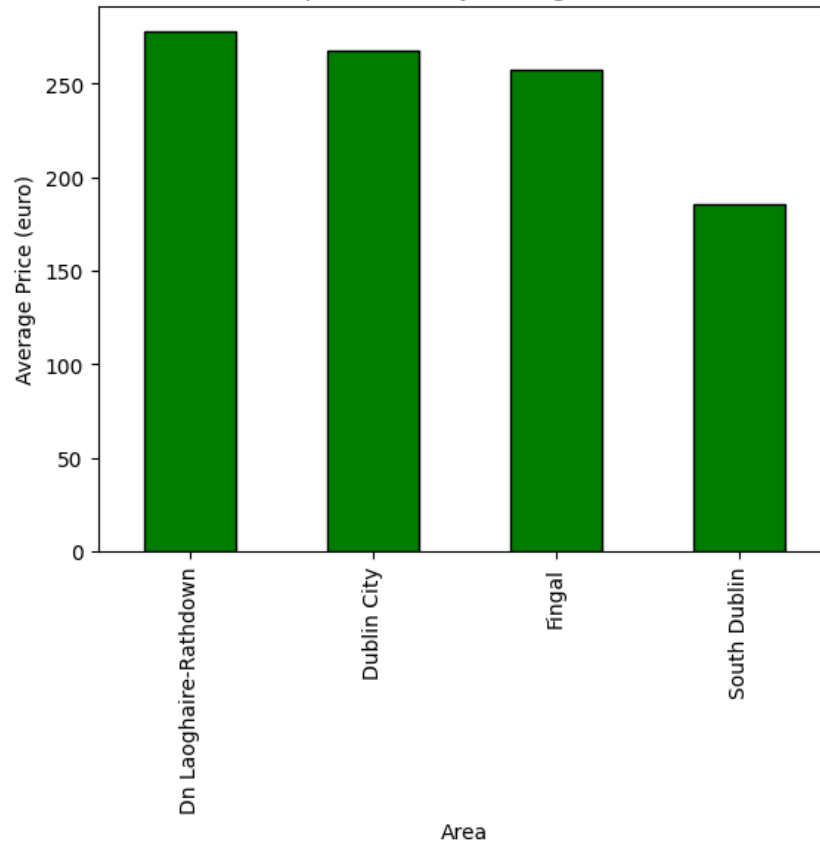
Dublin City 267.867053

Fingal 257.264056

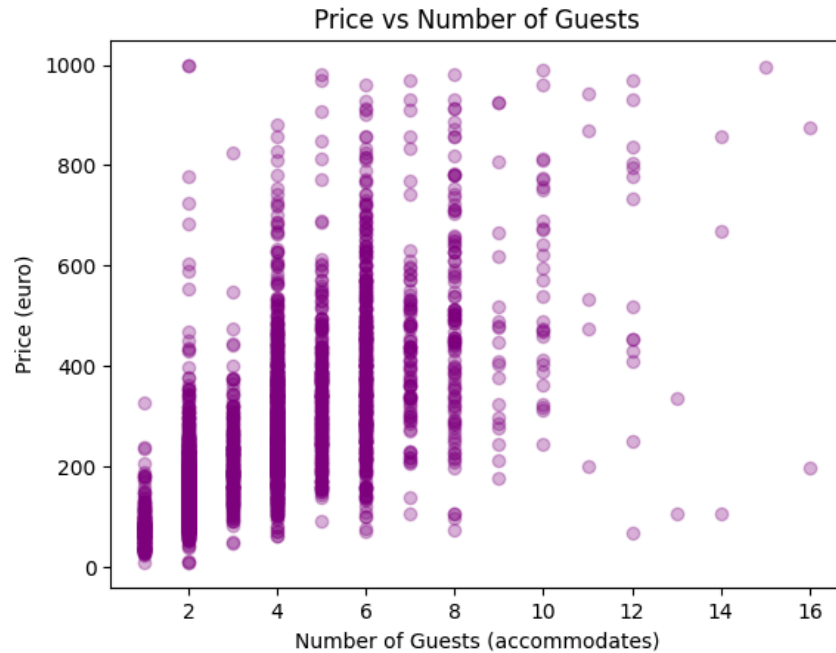
South Dublin 185.581486

Name: price, dtype: float64

Top 10 Areas by Average Price



```
1 # These codes are used for visualising to find does price go up when a place fits more number of people.
2
3 plt.scatter(df["accommodates"], df["price"], alpha=0.3, color="purple")
4 plt.title("Price vs Number of Guests")
5 plt.xlabel("Number of Guests (accommodates)")
6 plt.ylabel("Price (euro)")
7 plt.show()
```

```

1 # These codes are for prediction and using linear regression model.
2 X = df[["accommodates", "bedrooms", "number_of_reviews", # Using the variable x to denote the columns that is used in the prediction.
3         "minimum_nights", "availability_365"]] # Using the variable y to denote the column that is needed for the prediction,
4 y = df["price"]
5
6 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # Splitting the data into 80 percent for training the model and 20 percent
7
8 model = LinearRegression() # Created the model and using it in the training data.
9 model.fit(X_train, y_train)
10
11 predictions = model.predict(X_test) # Using the trained model to predict prices for the test data.
12
13 r2 = model.score(X_test, y_test) # These codes are for checking how good is the model if the score is between 0 and 1 if the value is higher it is better.
14 rmse = np.sqrt(np.mean((y_test - predictions) ** 2)) # If the average error is lower the better.
15 print("\nModel R-squared score:", round(r2, 3))
16 print("Model RMSE (average error in euro):", round(rmse, 2))
17

```

Model R-squared score: 0.438
Model RMSE (average error in euro): 130.51

```

1 # These codes are done to show the actual price to the predicted price.
2

```

```
3 results = pd.DataFrame({"Actual Price": y_test.values, "Predicted Price": predictions.round(2)})
```